คลังข้อสอบอัตนัยเว็บแอปพลิเคชัน บทที่ 1 ถึง 7

105 ข้อ แยกบท พร้อมเฉลยท้ายบทและจุดกลับไปอ่านสรุป

เล่มนี้ใช้คู่กับ สรุปสอบเว็บแอปพลิเคชัน_บท1-7 ฉบับ 20 หน้า โดยเลขหน้าอ้างอิงเป็นหน้าของ PDF สรุป ไม่ใช่เลขหน้าในคลังข้อสอบนี้

เลือกฝึกอย่างไรเมื่อเวลาเหลือน้อย

แต่ละบทมี 15 ข้อ ข้อ 1–10 ดัดแปลงประเด็น pretest โดยตรง รวม 70 ข้อ ส่วนข้อ 11–15 เป็นคำถามประยุกต์ เปรียบเทียบสถานการณ์ หรืออ่านโค้ด รวม 35 ข้อ การเน้นกลุ่มแรกหมายถึงตรงกับเนื้อหาที่เห็นใน pretest ไม่ใช่มีข้อมูลยืนยันความถี่ที่อาจารย์จะออก

ทำข้อที่ตอบไม่ได้ก่อนอ่านเฉลย เขียนเป็นประโยคที่มีความหมาย หลักการ ตัวอย่าง และเหตุผลตามโจทย์ ใช้คำต่างจากเฉลยได้หากใจความถูก หากทำข้อสั้นให้เวลาประมาณ 3–5 นาที ข้อโค้ดหรือวิเคราะห์ให้ 6–8 นาที

บท	กลับไปอ่านสรุป	จำนวนข้อ
1 HTTP และสถาปัตยกรรมเว็บ	หน้า 2–3	15
2 HTML CSS และ DOM	หน้า 4–5	15
3 JavaScript พื้นฐาน	หน้า 6–7	15
4 SPA และ Component	หน้า 8–9	15
5 Node.js และ Asynchronous Programming	หน้า 10–11	15
6 ฐานข้อมูลเชิงสัมพันธ์และ SQL	หน้า 12–14	15
7 MongoDB และฐานข้อมูลแบบ Document	หน้า 15–17	15

รูปแบบคำตอบที่ใช้ตรวจตนเอง

แนวตอบเป็นตัวอย่างคำตอบที่ถูกตามหลัก ไม่จำเป็นต้องท่องทุกคำ ส่วน “ประเด็นที่ควรครบ” ช่วยตรวจว่าเขียนใจความสำคัญแล้วหรือยัง ไม่ใช่เกณฑ์คะแนนทางการของอาจารย์

ขอบเขตเฉพาะบท 1–7 ไม่รวม Flask และ pretest ชุดที่ 8 คำถามทั้งหมดเป็นแนวฝึกที่สร้างจาก pretest และสรุปที่มี ไม่ใช่ข้อสอบจริงหรือคำรับรองว่าจะออกครบ

บทที่ 1 โจทย์เรื่อง HTTP และสถาปัตยกรรมเว็บ

อ่านประกอบจากสรุปหน้า 2–3 | เฉลยอยู่หลังโจทย์ทั้ง 15 ข้อของบทนี้

ข้อ 1 ถึง 10 ประเด็นตรงกับ pretest

1.1 HTTP response body คืออะไร ยกตัวอย่างข้อมูลที่อยู่ในส่วนนี้ และแยกจาก header

1.2 หากต้องการอ่านรายชื่อสินค้าจาก server ควรใช้ HTTP method ใด เพราะเหตุใด และต่างจาก POST อย่างไร

1.3 อธิบายลำดับตั้งแต่ browser ขอหน้าเว็บจนแสดงผล HTML

1.4 HTTPS เพิ่มความสามารถอะไรจาก HTTP และทำให้ HTTP จำสถานะผู้ใช้ได้เองหรือไม่

1.5 HTTP status code กลุ่ม 2xx หมายถึงอะไร และต่างจาก 4xx กับ 5xx อย่างไร

1.6 แยกส่วนของ URL https://shop.example/products?page=2#details และระบุส่วนที่บอกโพรโทคอล

1.7 HTTP เป็น stateless หมายถึงอะไร เหตุใดเว็บไซต์จึงยังจำการเข้าสู่ระบบได้

1.8 อธิบายบทบาท client และ server พร้อมตัวอย่างการเปิดหน้าแสดงสินค้า

1.9 Content-Type อยู่ส่วนใดของ HTTP message และมีหน้าที่อะไร

1.10 เมื่อ browser ได้ HTTP 404 หมายความว่าอะไร สรุปได้หรือไม่ว่า server ล่มหรือผู้ใช้ไม่มีสิทธิ์

ข้อ 11 ถึง 15 ฝึกประยุกต์และเชื่อมความเข้าใจ

1.11 หน้า /products แสดง 200 แต่รูป /cover.png แสดง 404 อธิบายว่าเหตุใดหน้าเว็บยังเปิดได้แต่รูปหาย

1.12 Server ตอบ Content-Type: application/json และ body เป็น {"name":"Book"}
จงอธิบายหน้าที่ของสองส่วนนี้

1.13 ผู้ใช้เปิดหน้าบัญชีได้หลัง login จงอธิบายว่า session เกี่ยวข้องกับ HTTP stateless อย่างไร

1.14 เปรียบเทียบความหมายของ request กับ response โดยระบุผู้ส่งและข้อมูลสำคัญอย่างละสองส่วน

1.15 อธิบายว่าการรับข้อมูลจาก server กับการ render เป็นหน้าที่เดียวกันหรือไม่

เฉลยท้ายบทที่ 1 ข้อ 1 ถึง 5

ทวนจากสรุปหน้า 2–3 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 1.1

แนวตอบ: Body คือเนื้อหาจริงที่ server ส่งกลับ เช่นเอกสาร HTML ข้อมูล JSON หรือรูปภาพ ส่วน header เป็นข้อมูลประกอบที่อธิบายข้อความ เช่น Content-Type บอกชนิดเนื้อหา บาง response อาจไม่มี body จึงไม่ควรสรุปว่าทุก response ต้องมี HTML เสมอ

ประเด็นที่ควรครบ: เนื้อหาจริง • HTML/JSON • header เป็นข้อมูลประกอบ

ข้อ 1.2

แนวตอบ: ใช้ GET เพราะมีความหมายสำหรับเรียกอ่านทรัพยากร เช่น GET /products ส่วน POST ใช้ส่งข้อมูลให้ server ประมวลผลและมักใช้สร้างรายการใหม่ เช่นส่งข้อมูลคำสั่งซื้อ ความหมายของ method บอกเจตนาของคำขอ ไม่ใช่ชนิดข้อมูลที่ได้รับกลับ

ประเด็นที่ควรครบ: GET อ่าน • POST ส่งหรือสร้าง • ยกตัวอย่างคำขอ

ข้อ 1.3

แนวตอบ: Browser ติดต่อ server และส่ง HTTP request จากนั้นรับ response ซึ่งอาจมี HTML เป็น body Browser อ่าน HTML สร้าง DOM โหลด CSS รูปและ JavaScript ที่จำเป็น แล้วจัดวางและวาดหน้า การ render อาจเกิดเป็นช่วงระหว่างโหลด แต่โดยทั่วไปต้องได้รับข้อมูลที่เกี่ยวข้องก่อนจึงแสดงได้

ประเด็นที่ควรครบ: request → response → อ่าน HTML → DOM → render

ข้อ 1.4

แนวตอบ: HTTPS คือ HTTP ที่สื่อสารผ่าน TLS ซึ่งช่วยเข้ารหัสข้อมูลระหว่างทาง ตรวจความครบถ้วนถูกต้องของข้อมูล และตรวจตัวตนของ server ผ่านใบรับรอง การใช้ HTTPS ไม่ทำให้ HTTP กลายเป็น stateful การจดจำผู้ใช้ยังต้องใช้กลไกเพิ่มเติม เช่น session/cookie/token

ประเด็นที่ควรครบ: TLS • เข้ารหัสและตรวจสอบ • ยัง stateless

ข้อ 1.5

แนวตอบ: 2xx หมายถึงคำขอสำเร็จ เช่น 200 OK กลุ่ม 4xx เกี่ยวกับปัญหาคำขอฝั่ง client เช่น 404 ไม่พบทรัพยากร ส่วน 5xx หมายถึง server ไม่สามารถทำคำขอให้สำเร็จ เช่น 500 Internal Server Error ต้องอ่านรหัสย่อยเพื่อรู้สาเหตุที่เจาะจง

ประเด็นที่ควรครบ: 2xx สำเร็จ • 4xx คำขอ • 5xx server

เฉลยท้ายบทที่ 1 ข้อ 6 ถึง 10

ทวนจากสรุปหน้า 2–3 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 1.6

แนวตอบ: Scheme คือ https และเป็นส่วนที่บอกโพรโทคอล Host คือ shop.example Path คือ /products Query string คือ page=2 ส่วน fragment คือ details โดยทั่วไป fragment ใช้ใน browser และไม่ถูกส่งเป็นส่วนของ HTTP request ไปยัง server

ประเด็นที่ควรครบ: scheme • host • path • query • fragment

ข้อ 1.7

แนวตอบ: HTTP ไม่จำสถานะการสนทนาระหว่าง request ให้เอง
แต่ละคำขอต้องมีข้อมูลที่ทำให้ระบบประมวลผลได้ เว็บไซต์ใช้กลไกเพิ่มเติม เช่น browser ส่ง cookie ที่มี session identifier ให้ server ใช้ค้น session หรือส่ง token เพื่อระบุผู้ใช้ Stateless ไม่ได้ห้าม server เก็บฐานข้อมูล

ประเด็นที่ควรครบ: ไม่จำอัตโนมัติ • request มีข้อมูลพอ • cookie/session/token

ข้อ 1.8

แนวตอบ: Client เป็นฝ่ายขอใช้บริการ เช่น browser ส่ง request ขอสินค้า Server รับคำขอ ประมวลผล อาจอ่านฐานข้อมูล แล้วส่ง response กลับ Browser จึงนำข้อมูลไปแสดงผล
ทั้งสองบทบาทไม่จำเป็นต้องทำงานบนเครื่องเดียวกัน

ประเด็นที่ควรครบ: client ขอ • server ประมวลผลและตอบ • browser แสดง

ข้อ 1.9

แนวตอบ: Content-Type เป็น header ใช้อธิบาย media type ของ body เช่น application/json บอกว่าเนื้อหาเป็น JSON หรือ text/html บอกว่าเป็น HTML ตัว header ไม่ใช่เนื้อหาสินค้าหรือข้อความ HTML จริง
แต่ช่วยผู้รับตีความเนื้อหาที่ส่งมา

ประเด็นที่ควรครบ: header • ชนิดเนื้อหา • ตัวอย่าง media type

ข้อ 1.10

แนวตอบ: 404 หมายถึงไม่พบทรัพยากรที่ร้องขอ เช่น URL ผิดหรือรายการถูกลบ การได้รับ 404 แสดงว่าได้รับ HTTP response แล้ว จึงสรุปไม่ได้ว่า server ล่ม และไม่ควรตีความว่าขาดสิทธิ์เสมอไป เพราะรหัสนี้บอกหลัก ๆ ว่าไม่พบทรัพยากรตามที่ขอ

ประเด็นที่ควรครบ: ไม่พบ resource • ได้ response แล้ว • ไม่สรุปสาเหตุเกินรหัส

เฉลยท้ายบทที่ 1 ข้อ 11 ถึง 15

ทวนจากสรุปหน้า 2–3 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 1.11

แนวตอบ: HTML กับรูปเป็นทรัพยากรคนละรายการ Browser ได้ HTML สำเร็จจึงแสดงโครงหน้าได้ แล้วส่ง request เพิ่มตาม URL ของรูป หากรูปได้ 404 จะโหลดรูปนั้นไม่ได้ ต้องตรวจ path หรือไฟล์รูป การสำเร็จของหน้า HTML ไม่รับประกันว่าทรัพยากรที่อ้างถึงทุกชิ้นสำเร็จด้วย

ประเด็นที่ควรครบ: คนละ request • HTML สำเร็จ • รูปไม่พบ

ข้อ 1.12

แนวตอบ: Content-Type เป็น metadata ระบุว่า body อยู่ในรูป JSON ส่วน {"name":"Book"} คือข้อมูลจริงที่มี field name และค่า Book Client ใช้ข้อมูลประกอบช่วยเลือกวิธีอ่าน body แล้วอาจนำชื่อสินค้าไปแสดงใน DOM

ประเด็นที่ควรครบ: metadata • ข้อมูลจริง • การตีความและแสดงผล

ข้อ 1.13

แนวตอบ: หลัง login server อาจสร้าง session และส่ง identifier ให้ browser เก็บใน cookie
คำขอหน้าบัญชีครั้งถัดไปแนบ cookie มา Server จึงรู้ว่าจะใช้ session ใด
นี่เป็นกลไกของแอปที่สร้างความต่อเนื่องบน HTTP ซึ่งไม่ได้จำการสนทนาเอง

ประเด็นที่ควรครบ: สร้าง session • ส่ง identifier • แนบในคำขอถัดไป

ข้อ 1.14

แนวตอบ: Request ส่งโดย client เพื่อขอให้ server ทำงาน มี method และเป้าหมาย เช่น GET /products รวมทั้ง headers และอาจมี body ส่วน response ส่งโดย server เพื่อแจ้งผล มี status code เช่น 200 และ body เช่น JSON พร้อม headers อธิบายผลตอบ

ประเด็นที่ควรครบ: ผู้ส่ง • method/เป้าหมาย • status/ผลตอบ

ข้อ 1.15

แนวตอบ: เป็นคนละขั้น การรับข้อมูลคือการสื่อสารผ่าน request/response ส่วนการ render คือ browser ตีความเนื้อหาและรูปแบบเพื่อจัดวางและวาดบนจอ Server อาจส่ง HTML ที่เตรียมแล้ว แต่ browser ยังต้องแสดงผล HTML นั้น และอาจส่งข้อมูล JSON ให้ JavaScript สร้าง UI ต่อได้

ประเด็นที่ควรครบ: สื่อสาร • ตีความและวาด • HTML/JSON

บทที่ 2 โจทย์เรื่อง HTML CSS และ DOM

อ่านประกอบจากสรุปหน้า 4–5 | เฉลยอยู่หลังโจทย์ทั้ง 15 ข้อของบทนี้

ข้อ 1 ถึง 10 ประเด็นตรงกับ pretest

2.1 HTML ทำหน้าที่หลักอะไร และต่างจาก CSS กับ JavaScript อย่างไร

2.2 เหตุใดหัวข้อหลักจึงควรใช้ <h1> และการทำให้ข้อความใหญ่ด้วย CSS แทน <h1> เหมือนกันหรือไม่

2.3 Semantic HTML คืออะไร และมีประโยชน์ต่อใครบ้าง ยกตัวอย่างสองแท็ก

2.4 เขียน CSS selector เพื่อเลือก element ที่มี id="main" พร้อมอธิบายความหมาย

2.5 เขียน selector เพื่อเลือกทุก element ที่มี class="card" และอธิบายเหตุใดใช้กับหลาย element ได้

2.6 อธิบาย CSS box model จากด้านในออกด้านนอก พร้อมแยก padding กับ margin

2.7 display: flex เหมาะกับงานใด อธิบาย main axis กับ cross axis

2.8 DOM คืออะไร และเหตุใด JavaScript จึงเปลี่ยนข้อความบนหน้าเว็บได้

2.9 document.querySelector() ทำหน้าที่อะไร ถ้าตรงหลาย element หรือไม่พบจะได้อะไร

2.10 เหตุใดควรแยก HTML กับ CSS ตามหน้าที่ ยกสถานการณ์ที่ช่วยให้แก้ไขงานง่ายขึ้น

ข้อ 11 ถึง 15 ฝึกประยุกต์และเชื่อมความเข้าใจ

2.11 ปุ่มมีข้อความชิดขอบและติดกับปุ่มข้าง ๆ ต้องแก้ padding หรือ margin อย่างไร

2.12 เขียนตัวอย่าง JavaScript ให้กด #buy แล้วเปลี่ยนข้อความ #message เป็น “เลือกแล้ว” สมมติ element มีอยู่แล้ว

2.13 เปรียบเทียบ HTML กับ DOM โดยยกตัวอย่างปุ่มเดียวกัน

2.14 ใน flex-direction: column คำสั่ง justify-content กับ align-items จัดแนวใด เพราะอะไร

2.15 อธิบาย responsive design และความหมายของ Bootstrap col-12 col-md-6

เฉลยท้ายบทที่ 2 ข้อ 1 ถึง 5

ทวนจากสรุปหน้า 4–5 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 2.1

แนวตอบ: HTML กำหนดโครงสร้างและความหมาย เช่นหัวเรื่อง ย่อหน้า และปุ่ม CSS กำหนดการแสดงผล เช่นสี ระยะห่าง และ layout ส่วน JavaScript จัดการพฤติกรรม เช่นกดปุ่มแล้วเปลี่ยนข้อความ
ทั้งสามทำงานร่วมกันโดยมีหน้าที่ต่างกัน

ประเด็นที่ควรครบ: โครงสร้าง • หน้าตา • พฤติกรรม

ข้อ 2.2

แนวตอบ: <h1> สื่อว่าข้อความเป็นหัวข้อระดับสูงสุดของเนื้อหา การขยาย font-size
ให้ข้อความทั่วไปทำให้ดูใหญ่แต่ไม่ได้ให้ความหมายเป็น heading เท่ากัน จึงเลือกแท็กตามโครงสร้างและใช้ CSS คุมรูปลักษณ์ ไม่เลือก heading เพราะขนาดตัวอักษรเพียงอย่างเดียว

ประเด็นที่ควรครบ: ความหมาย heading • ขนาดไม่เท่าความหมาย • CSS คุมหน้าตา

ข้อ 2.3

แนวตอบ: Semantic HTML ใช้ element ที่สื่อความหมายตรงหน้าที่ เช่น <nav> เป็น navigation และ <main> เป็นเนื้อหาหลัก ช่วยคนอ่านโค้ด browser ระบบค้นหา และ screen reader เข้าใจโครงสร้าง โดยไม่ได้ทำให้เลิกใช้ CSS หรือ JavaScript

ประเด็นที่ควรครบ: แท็กตรงหน้าที่ • nav/main • browser/search/screen reader

ข้อ 2.4

แนวตอบ: ใช้ #main เพราะเครื่องหมาย # ระบุ selector ตาม id เช่น #main { color: black; } ใช้กำหนดสีให้ element เป้าหมาย id ควรไม่ซ้ำในเอกสาร ต่างจาก .main ซึ่งเลือก class ชื่อ main

ประเด็นที่ควรครบ: #main • id • ต่างจาก class

ข้อ 2.5

แนวตอบ: ใช้ .card เพราะจุดนำหน้าหมายถึง class selector หลาย element มี class card ร่วมกันได้ จึงใช้รูปแบบ CSS เดียวกับการ์ดสินค้าหลายใบได้ ต่างจาก id ที่ควรระบุ element แบบไม่ซ้ำในหน้า

ประเด็นที่ควรครบ: .card • class ใช้ร่วมกัน • รูปแบบใช้ซ้ำ

เฉลยท้ายบทที่ 2 ข้อ 6 ถึง 10

ทวนจากสรุปหน้า 4–5 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 2.6

แนวตอบ: เรียงเป็น content, padding, border, margin Content คือเนื้อหา Padding เว้นภายในระหว่างเนื้อหากับขอบ Border เป็นเส้นขอบ และ margin เว้นภายนอกระหว่างกล่องกับสิ่งรอบข้าง เช่นเพิ่ม padding ทำให้ข้อความปุ่มห่างขอบ ส่วนเพิ่ม margin ทำให้ปุ่มห่างปุ่มอื่น

ประเด็นที่ควรครบ: content → padding → border → margin • ตัวอย่าง

ข้อ 2.7

แนวตอบ: Flexbox เหมาะกับการจัดสมาชิกตามแกนหลัก เช่นแถวของปุ่ม Main axis เป็นแกนตาม flex-direction และ cross axis ตั้งฉากกับแกนหลัก เมื่อเป็น row แกนหลักเป็นแนวนอน แต่ถ้าเป็น column แกนหลักเปลี่ยนเป็นแนวตั้ง

ประเด็นที่ควรครบ: จัดสมาชิก • flex-direction • แกนเปลี่ยนได้

ข้อ 2.8

แนวตอบ: DOM เป็นตัวแทนเอกสารแบบต้นไม้ที่ browser สร้างจาก HTML แต่ละ element เป็นวัตถุหรือ node JavaScript เลือก node และแก้ property เช่น textContent ได้ Browser จึงสะท้อนการเปลี่ยนแปลงเป็นหน้าเว็บ DOM ไม่ใช่ฐานข้อมูลหรือโพรโทคอลเครือข่าย

ประเด็นที่ควรครบ: ต้นไม้เอกสาร • node/object • อ่านแก้ได้

ข้อ 2.9

แนวตอบ: querySelector รับ CSS selector แล้วคืน element แรกที่ตรงเงื่อนไข ถ้าไม่มีจะคืน null หากต้องการทั้งหมดใช้ querySelectorAll เช่น document.querySelector(".card") เลือกการ์ดแรก ไม่ใช่ทุกการ์ด

ประเด็นที่ควรครบ: CSS selector • ตัวแรก • null

ข้อ 2.10

แนวตอบ: HTML ดูแลโครงสร้าง ส่วน CSS ดูแลการนำเสนอ การแยกช่วยเปลี่ยนสีหรือระยะห่างจาก stylesheet ที่ใช้ร่วมกัน โดยไม่ต้องเปลี่ยนโครงสร้างเนื้อหาทุกหน้า เช่นแก้ .card ครั้งเดียวแล้วการ์ดทุกใบที่ใช้ class นี้เปลี่ยนตาม

ประเด็นที่ควรครบ: แยกหน้าที่ • ดูแลง่าย • stylesheet ใช้ร่วมกัน

เฉลยท้ายบทที่ 2 ข้อ 11 ถึง 15

ทวนจากสรุปหน้า 4–5 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 2.11

แนวตอบ: เพิ่ม padding เพื่อให้ข้อความห่างขอบภายในปุ่ม และเพิ่ม margin หรือ gap ของ parent เพื่อเว้นระยะระหว่างปุ่ม การเพิ่ม margin อย่างเดียวไม่ทำให้ข้อความด้านในห่าง border และการเพิ่ม padding ไม่ใช่การระบุช่องว่างภายนอกโดยตรง

ประเด็นที่ควรครบ: padding ภายใน • margin/gap ภายนอก • เหตุผล

ข้อ 2.12

```
const button = document.querySelector('#buy');
button.addEventListener('click', () => {
  document.querySelector('#message').textContent = 'เลือกแล้ว';
});
```

เริ่มจากเลือกปุ่ม ลงทะเบียน callback แล้วเปลี่ยนข้อความเมื่อเกิด click โค้ดนี้ต้องรันหลัง element ถูกสร้างหรือจัดการจังหวะโหลดให้ถูกต้อง

ประเด็นที่ควรครบ: เลือก element • click listener • textContent

ข้อ 2.13

แนวตอบ: HTML คือภาษาและเอกสารต้นทาง เช่น <button id="buy">ซื้อ</button> ส่วน DOM คือวัตถุ button ที่ browser สร้างจากเอกสารนั้น JavaScript เปลี่ยนวัตถุใน DOM ได้
จึงเปลี่ยนหน้าเว็บระหว่างใช้งานได้โดยไม่ต้องแก้ไฟล์ HTML ต้นฉบับบน server ทุกครั้ง

ประเด็นที่ควรครบ: เอกสารต้นทาง • วัตถุใน browser • เปลี่ยนระหว่างใช้งาน

ข้อ 2.14

แนวตอบ: เมื่อเป็น column main axis เป็นแนวตั้ง ดังนั้น justify-content จัดตามแนวตั้ง ส่วน align-items จัดตาม cross axis ซึ่งเป็นแนวนอน ต้องดู flex-direction ก่อน จึงไม่ควรท่องว่า justify-content จัดแนวนอนเสมอ

ประเด็นที่ควรครบ: column • main แนวตั้ง • cross แนวนอน

ข้อ 2.15

แนวตอบ: Responsive design ปรับ layout ให้ใช้ได้กับหลายขนาดจอ Bootstrap แบ่ง grid เป็น 12 ส่วน col-12 ให้เต็มแถวเป็นค่าฐาน ส่วน col-md-6 ให้กิน 6 จาก 12 ส่วนตั้งแต่ breakpoint md ขึ้นไป
จึงจัดสองคอลัมน์ได้เมื่อจอกว้างพอ เป็นความรู้เสริมจากสรุปหน้า 5

ประเด็นที่ควรครบ: ปรับตามจอ • 12 ส่วน • เต็มแถวแล้วครึ่งแถว

บทที่ 3 โจทย์เรื่อง JavaScript พื้นฐาน

อ่านประกอบจากสรุปหน้า 6–7 | เฉลยอยู่หลังโจทย์ทั้ง 15 ข้อของบทนี้

ข้อ 1 ถึง 10 ประเด็นตรงกับ pretest

3.1 เปรียบเทียบ let กับ const พร้อมยกตัวอย่างที่กำหนดค่าใหม่ได้และไม่ได้

3.2 เครื่องหมาย = กับ === ต่างกันอย่างไร และ 2 === "2" ให้ผลอะไร

3.3 Higher-order function คืออะไร ยกตัวอย่างและอธิบายบทบาท callback

3.4 map() ทำอะไร และต่างจาก push() อย่างไรทั้งผลต่อ array และค่าที่คืน

3.5 Block scope คืออะไร ตัวแปร let ที่ประกาศใน if ใช้นอกบล็อกได้หรือไม่

3.6 Object ใน JavaScript เก็บข้อมูลแบบใด ยกตัวอย่างและอ่านค่าหนึ่ง field

3.7 Event listener คืออะไร ใช้กับเหตุการณ์ประเภทใดได้ และทำงานตอนไหน

3.8 เขียนคำสั่งเพิ่ม click listener ให้ button โดยใช้ handler และอธิบายว่าเหตุใดไม่ใส่ handler()

3.9 เขียน arrow function ที่รับ x แล้วคืน x คูณ 2 ทั้งแบบสั้นและแบบมีวงเล็บปีกกา

3.10 try...catch ใช้ทำอะไร ยกตัวอย่างความผิดพลาดที่ควรจัดการ

ข้อ 11 ถึง 15 ฝึกประยุกต์และเชื่อมความเข้าใจ

3.11 const user={name:"Ann"}; user.name="Bee"; กับ user={name:"Bee"}; ต่างกันอย่างไร

3.12 const a=[1,2,3]; const b=a.map(x=>x*2); const n=a.push(4); จงบอกค่า a, b และ n

3.13 const f=x=>{x*2}; f(3) คืนอะไร และควรแก้ให้คืน 6 อย่างไร

3.14 const a=[10,20,30]; const x=a.shift(); const y=a.pop(); จงบอก x, y และ a หลังทำงาน

3.15 Callback ทุกตัวเป็น asynchronous หรือไม่ อธิบายโดยใช้ map กับ event listener

เฉลยท้ายบทที่ 3 ข้อ 1 ถึง 5

ทวนจากสรุปหน้า 6–7 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 3.1

แนวตอบ: let กำหนดค่าใหม่ได้ เช่น let n=1; n=2; ส่วน const ไม่ให้กำหนดค่าใหม่หลังประกาศ เช่น const n=1; n=2; ทำให้เกิดข้อผิดพลาด ทั้งสองมี block scope ถ้า const อ้าง object ยังแก้ property ภายในได้ เพราะไม่ได้เปลี่ยนการอ้างอิงของตัวแปร

ประเด็นที่ควรครบ: reassignment • block scope • property แก้ได้

ข้อ 3.2

แนวตอบ: = ใช้กำหนดค่าให้ตัวแปร ส่วน === เปรียบเทียบแบบไม่แปลงชนิด 2 === "2" ได้ false เพราะ 2 เป็น number แต่ "2" เป็น string แม้ตัวเลขที่อ่านดูเหมือนกัน ต่างจาก 2 === 2 ซึ่งได้ true

ประเด็นที่ควรครบ: กำหนดค่า • เปรียบเทียบไม่แปลงชนิด • false

ข้อ 3.3

แนวตอบ: เป็นฟังก์ชันที่รับฟังก์ชันอื่นเป็น argument หรือคืนฟังก์ชัน เช่น map รับ callback เพื่อกำหนดว่าจะแปลงสมาชิกแต่ละตัวอย่างไร ใน [1,2].map(x=>x*2) ฟังก์ชัน x=>x*2 เป็น callback ส่วน map เป็น higher-order function Callback ไม่จำเป็นต้อง asynchronous

ประเด็นที่ควรครบ: รับหรือคืนฟังก์ชัน • map • callback

ข้อ 3.4

แนวตอบ: map แปลงสมาชิกด้วย callback และคืน array ใหม่ เช่น [1,2].map(x=>x*2) ได้ [2,4] ส่วน push เพิ่มสมาชิกท้าย array เดิมและคืนความยาวใหม่ เช่น a=[1,2]; a.push(3) ทำให้ a เป็น [1,2,3] และคืน 3

ประเด็นที่ควรครบ: array ใหม่ • แก้ array เดิม • ความยาว

ข้อ 3.5

แนวตอบ: Block scope จำกัดการเข้าถึงตัวแปรไว้ในบล็อก { } ที่ประกาศและขอบเขตย่อยภายใน เช่น if(true){let x=1;} แล้วใช้ x ภายนอกไม่ได้ หากไม่มีตัวแปร x อื่นจะเกิด ReferenceError โดย let และ const มีลักษณะนี้

ประเด็นที่ควรครบ: ขอบเขตบล็อก • let/const • ภายนอกเข้าถึงไม่ได้

เฉลยท้ายบทที่ 3 ข้อ 6 ถึง 10

ทวนจากสรุปหน้า 6–7 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 3.6

แนวตอบ: Object เก็บข้อมูลเป็นคู่ key–value เช่น const user={name:"Ann",age:20}; key คือ name กับ age ส่วน value คือ "Ann" กับ 20 อ่านชื่อได้ด้วย user.name หรือ user["name"]
โครงสร้างนี้ไม่ได้จำกัดให้มีเพียงแถวและคอลัมน์

ประเด็นที่ควรครบ: key–value • ตัวอย่าง • อ่าน property

ข้อ 3.7

แนวตอบ: Event listener คือฟังก์ชันที่ลงทะเบียนให้ถูกเรียกเมื่อเกิดเหตุการณ์ เช่น click, input หรือ submit การลงทะเบียนไม่เท่ากับสั่งให้ทำงานทันที เมื่อเหตุการณ์ที่กำหนดเกิดขึ้นจึงเรียก handler เพื่อทำงานตอบสนอง

ประเด็นที่ควรครบ: ลงทะเบียน • เหตุการณ์ • handler ทำงานเมื่อเกิด event

ข้อ 3.8

แนวตอบ: ใช้ button.addEventListener("click", handler); เพื่อส่งตัวฟังก์ชันให้ระบบเรียกตอนคลิก หากเขียน handler() จะเรียกฟังก์ชันทันทีตอนลงทะเบียนและส่งค่าที่ฟังก์ชันคืนกลับไป ซึ่งมักไม่ใช่ listener ที่ต้องการ

ประเด็นที่ควรครบ: addEventListener • ส่งฟังก์ชัน • ไม่เรียกทันที

ข้อ 3.9

แนวตอบ: แบบสั้นคือ const double = x => x * 2; แบบมีปีกกาคือ const double = x => { return x * 2; }; แบบสั้นคืนค่านิพจน์โดยอัตโนมัติ แต่แบบบล็อกต้องมี return เมื่อต้องการคืนผลลัพธ์

ประเด็นที่ควรครบ: x => x * 2 • return ในบล็อก • ผลลัพธ์

ข้อ 3.10

แนวตอบ: try ครอบคำสั่งที่อาจโยน exception หากเกิดข้อผิดพลาดจะเข้า catch เพื่อจัดการ เช่นแสดงข้อความหรือส่งต่อข้อผิดพลาด ตัวอย่าง JSON.parse ที่ได้รับข้อความไม่ใช่ JSON อาจโยน exception การมี catch ไม่ได้ทำให้ข้อมูลผิดถูกแก้เอง

ประเด็นที่ควรครบ: exception • catch • จัดการอย่างเหมาะสม

เฉลยท้ายบทที่ 3 ข้อ 11 ถึง 15

ทวนจากสรุปหน้า 6–7 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 3.11

แนวตอบ: คำสั่งแรกเปลี่ยน property ของ object เดิมจึงทำได้ ส่วนคำสั่งหลังพยายามเปลี่ยนการอ้างอิงของตัวแปร const ไปเป็น object ใหม่จึงทำไม่ได้ const ไม่ได้ทำให้ object เดิมกลายเป็นข้อมูลที่เปลี่ยนไม่ได้ทั้งหมด

ประเด็นที่ควรครบ: property เดิม • อ้างอิงใหม่ • const

ข้อ 3.12

แนวตอบ: หลัง map ค่า b เป็น [2,4,6] และ a ยังเป็น [1,2,3] หลัง push ค่า a เป็น [1,2,3,4] ส่วน n เป็น 4 ซึ่งคือความยาวใหม่ b ยังคงเป็น [2,4,6] การใช้ const ไม่ห้ามเพิ่มสมาชิกใน array เดิม

ประเด็นที่ควรครบ: a=[1,2,3,4] • b=[2,4,6] • n=4

ข้อ 3.13

แนวตอบ: คืน undefined เพราะภายในบล็อกไม่มี return แก้เป็น x=>{return x*2;} หรือ x=>x*2 แบบนิพจน์สั้นแล้ว f(3) จึงคืน 6 การคำนวณนิพจน์อย่างเดียวไม่ได้ส่งค่ากลับจากฟังก์ชันบล็อก

ประเด็นที่ควรครบ: undefined • ขาด return • สองวิธีแก้

ข้อ 3.14

แนวตอบ: shift เอาสมาชิกแรกออกและคืน 10 ทำให้ a เป็น [20,30] จากนั้น pop เอาสมาชิกท้ายออกและคืน 30 ทำให้ a เหลือ [20] ทั้งสองคำสั่งเปลี่ยน array เดิม

ประเด็นที่ควรครบ: x=10 • y=30 • a=[20]

ข้อ 3.15

แนวตอบ: ไม่ใช่ Callback หมายถึงฟังก์ชันที่ส่งให้โค้ดอีกส่วนเรียก map เรียก callback ระหว่างการแปลงสมาชิกแบบ synchronous ส่วน event listener ถูกเรียกเมื่อเหตุการณ์เกิดในภายหลัง จึงต้องแยกแนวคิด “ส่งฟังก์ชัน” ออกจาก “งานที่รอทำภายหลัง”

ประเด็นที่ควรครบ: callback ไม่เท่ากับ async • map • event

บทที่ 4 โจทย์เรื่อง SPA และ Component

อ่านประกอบจากสรุปหน้า 8–9 | เฉลยอยู่หลังโจทย์ทั้ง 15 ข้อของบทนี้

ข้อ 1 ถึง 10 ประเด็นตรงกับ pretest

4.1 SPA คืออะไร และคำว่า single page หมายถึงมีเพียง URL เดียวหรือไม่

4.2 Component ใน React หรือ Vue มีแนวคิดอย่างไร ยกตัวอย่างหนึ่ง component ที่นำกลับมาใช้ได้

4.3 State หมายถึงอะไร และสัมพันธ์กับการ render UI อย่างไร

4.4 Props ใช้ทำอะไร ข้อมูลเดินทางในทิศใดโดยทั่วไป

4.5 Client-side routing ทำหน้าที่อะไรเมื่อผู้ใช้เปลี่ยนหน้าภายใน SPA

4.6 History API ช่วย SPA อย่างไร และเปลี่ยน URL แล้ว UI เปลี่ยนเองเสมอหรือไม่

4.7 Component-based architecture มีข้อดีด้านการพัฒนาและทดสอบอย่างไร

4.8 หากออกแบบ SPA ไม่ดี อาจเกิดปัญหาใดเกี่ยวกับการโหลดครั้งแรก SEO และ accessibility

4.9 เปรียบเทียบ server-side route กับ client-side route โดยระบุสถานที่ทำงานและผลที่เกิดขึ้น

4.10 เหตุใด framework จึงใช้ Virtual DOM หรือ reactive system

ข้อ 11 ถึง 15 ฝึกประยุกต์และเชื่อมความเข้าใจ

4.11 หน้าร้านส่งชื่อและราคาให้ ProductCard แต่ผู้ใช้เลือกจำนวนเอง อะไรเป็น props และอะไรเป็น state

4.12 อธิบายลำดับเมื่อผู้ใช้คลิกสินค้าหนึ่งชิ้นใน SPA จนเห็นรายละเอียด

4.13 ทำไม SPA ยังต้องมี server ได้ และการไม่มี full-page reload หมายถึงไม่ใช้ HTTP หรือไม่

4.14 ถ้า child ต้องการเปลี่ยนข้อมูลที่รับมาทาง props ควรออกแบบการสื่อสารอย่างไร

4.15 หากเปลี่ยน URL ด้วย History API แล้วหน้ายังแสดง component เดิม จงอธิบายสาเหตุเชิงแนวคิด

เฉลยท้ายบทที่ 4 ข้อ 1 ถึง 5

ทวนจากสรุปหน้า 8–9 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 4.1

แนวตอบ: SPA โหลดเอกสารหรือโครงหลักของแอปก่อน แล้ว JavaScript เปลี่ยน UI ตามการใช้งานโดยไม่ต้องโหลดเอกสาร HTML ใหม่ทั้งหน้าทุกครั้ง จึงยังมีหลาย URL ผ่าน client-side routing ได้ และมี backend ได้ ไม่ได้หมายถึงมี HTML เพียงแท็กเดียว

ประเด็นที่ควรครบ: โหลดโครงหลัก • เปลี่ยน UI • หลาย URL ได้

ข้อ 4.2

แนวตอบ: Component แบ่ง UI เป็นส่วนย่อยที่มีหน้าที่เฉพาะและนำมาประกอบกัน เช่น ProductCard แสดงรูป ชื่อ และราคาของสินค้า ใช้โครงเดิมกับสินค้าหลายชิ้นโดยส่งข้อมูลต่างกัน ช่วยแยกความรับผิดชอบและใช้ซ้ำ

ประเด็นที่ควรครบ: หน่วยย่อย UI • ประกอบได้ • ProductCard

ข้อ 4.3

แนวตอบ: State เป็นข้อมูลที่ระบบหรือ component จัดการและเปลี่ยนได้ระหว่างใช้งาน เช่นจำนวนสินค้าที่เลือก เมื่ออัปเดตผ่านกลไกของ framework ระบบจะปรับการแสดงผลให้สอดคล้อง
การเปลี่ยนตัวแปรทั่วไปไม่ได้รับประกันว่าจะอัปเดต UI อัตโนมัติในทุก framework

ประเด็นที่ควรครบ: ข้อมูลเปลี่ยนได้ • กลไก framework • UI ตามข้อมูล

ข้อ 4.4

แนวตอบ: Props ใช้ส่งข้อมูลหรือการตั้งค่าเข้า component โดยทั่วไปจาก parent ไป child เช่นหน้าร้านส่ง name และ price ให้ ProductCard Child ใช้ค่าที่ได้รับเพื่อแสดงผล และตามหลักทั่วไปไม่แก้ props ของตัวเองโดยตรง แต่แจ้งผู้รับผิดชอบข้อมูลให้เปลี่ยนค่า

ประเด็นที่ควรครบ: parent → child • ข้อมูลส่งเข้า • ไม่แก้โดยตรง

ข้อ 4.5

แนวตอบ: Client router ใช้ URL เลือกมุมมองหรือ component ที่แสดงใน browser ทำให้เปลี่ยนหน้าภายในแอปโดยไม่จำเป็นต้องขอ HTML ใหม่ทั้งหน้า อย่างไรก็ดีอาจยังต้องส่ง HTTP request ไปขอข้อมูลหรือโค้ดเพิ่มเติม

ประเด็นที่ควรครบ: URL • เลือก UI • ไม่จำเป็นต้อง reload เอกสาร

เฉลยท้ายบทที่ 4 ข้อ 6 ถึง 10

ทวนจากสรุปหน้า 8–9 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 4.6

แนวตอบ: History API ช่วยจัดการ URL และประวัติการนำทาง เช่น pushState เพิ่มรายการประวัติโดยไม่ reload หน้า แต่ไม่ได้สร้าง UI ใหม่ให้เอง โค้ด router ต้องเลือก component และจัดการการแสดงผลต่อให้สอดคล้องกับ URL

ประเด็นที่ควรครบ: URL/ประวัติ • ไม่ reload • router จัด UI

ข้อ 4.7

แนวตอบ: แบ่งความรับผิดชอบเป็นส่วนเล็กที่เข้าใจง่าย เช่นปุ่มค้นหาและการ์ดสินค้าแยกกัน ทดสอบแต่ละ component ได้สะดวก นำกลับมาใช้หลายจุด และปรับปรุงโครงเดียวให้หลายหน้ารับผลได้ แต่ยังต้องออกแบบขอบเขตและข้อมูลระหว่าง component ให้เหมาะสม

ประเด็นที่ควรครบ: แยกหน้าที่ • ทดสอบ • ใช้ซ้ำ

ข้อ 4.8

แนวตอบ: หาก bundle ใหญ่ browser อาจต้องโหลดและประมวลผล JavaScript มากก่อนใช้หน้าได้ SEO อาจซับซ้อนเมื่อเนื้อหาต้องรอ JavaScript ส่วน accessibility ต้องจัดการ focus ลำดับ DOM และการเปลี่ยนเนื้อหาให้ผู้ใช้เครื่องมือช่วยรับรู้ ปัญหาเหล่านี้จัดการได้ ไม่ใช่ข้อห้ามของ SPA

ประเด็นที่ควรครบ: bundle • ค้นพบเนื้อหา • เข้าถึงและรับรู้การเปลี่ยนหน้า

ข้อ 4.9

แนวตอบ: Server route ทำงานที่ server จับคู่คำขอกับ handler เพื่อส่งข้อมูลหรือหน้าเว็บ ส่วน client route ทำงานใน browser เลือก UI ตาม URL โดยมักไม่ reload ทั้งเอกสาร ทั้งสองใช้งานร่วมกันได้ เช่น client route เปิดหน้ารายละเอียดแล้วขอข้อมูลจาก server

ประเด็นที่ควรครบ: server handler • browser UI • ทำงานร่วมกัน

ข้อ 4.10

แนวตอบ: ช่วยติดตามและประสานการเปลี่ยนแปลงของข้อมูลกับ UI เพื่อจัดการการอัปเดตอย่างเป็นระบบ เช่นคำนวณส่วนที่ต้องเปลี่ยนแล้วปรับ DOM กลไกแต่ละ framework ต่างกัน และไม่ควรสรุปว่าเร็วกว่าการแก้ DOM โดยตรงทุกกรณี

ประเด็นที่ควรครบ: ข้อมูลกับ UI • จัดการการเปลี่ยนแปลง • ไม่รับประกันทุกกรณี

เฉลยท้ายบทที่ 4 ข้อ 11 ถึง 15

ทวนจากสรุปหน้า 8–9 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 4.11

แนวตอบ: ชื่อและราคาที่ parent ส่งเข้า ProductCard เป็น props ส่วนจำนวนที่กำลังเลือกเป็น state ของ component หรือของส่วนที่รับผิดชอบตะกร้า เมื่อกดเพิ่ม จำนวนเปลี่ยนและ UI ควรเปลี่ยนตาม หากหลาย component ใช้จำนวนเดียวกันอาจย้าย state ไป parent ร่วม

ประเด็นที่ควรครบ: ชื่อราคา props • จำนวน state • ผู้รับผิดชอบข้อมูล

ข้อ 4.12

แนวตอบ: Event handler รับการคลิก Client router เปลี่ยนหรือประมวลผล URL แล้วเลือก component รายละเอียด หากข้อมูลยังไม่มีจึงขอจาก server เมื่อได้รับผลให้อัปเดต state แล้ว framework render ข้อมูลใหม่ ลำดับการขอข้อมูลอาจต่างกันตามแอป แต่ไม่จำเป็นต้องโหลด HTML ทั้งหน้า

ประเด็นที่ควรครบ: click • route • ขอข้อมูล • state/UI

ข้อ 4.13

แนวตอบ: SPA อาจใช้ server สำหรับข้อมูล ธุรกิจ และฐานข้อมูล แม้เปลี่ยน UI ใน browser โดยไม่ full-page reload แต่ยังส่ง HTTP request เพื่อขอหรือบันทึกข้อมูลได้ คำว่า SPA อธิบายรูปแบบการทำงานของหน้าแอป ไม่ได้ยกเลิกการสื่อสาร client–server

ประเด็นที่ควรครบ: backend ยังมี • HTTP ยังใช้ • แยก UI กับข้อมูล

ข้อ 4.14

แนวตอบ: ตามหลักทั่วไป child ส่ง event หรือเรียก callback ที่ได้รับเพื่อแจ้ง parent หรือเจ้าของ state ให้เปลี่ยนข้อมูล จากนั้นส่ง props ค่าใหม่กลับลงมา เช่น ProductCard แจ้ง add-to-cart แล้วหน้าร้านอัปเดตตะกร้า ช่วยให้รู้ว่าข้อมูลถูกเปลี่ยนที่ใด

ประเด็นที่ควรครบ: event/callback • เจ้าของ state เปลี่ยน • props ใหม่

ข้อ 4.15

แนวตอบ: History API จัดการ URL และประวัติ ไม่ได้จับคู่ route กับ component ให้ทั้งหมดเอง ต้องมีโค้ด router รับการเปลี่ยนเส้นทางและอัปเดต UI หากเปลี่ยน URL อย่างเดียวโดยไม่เรียกขั้นตอนนี้ UI จึงอาจคงเดิม

ประเด็นที่ควรครบ: URL ไม่ใช่ UI • ต้องมี router • อัปเดตมุมมอง

บทที่ 5 โจทย์เรื่อง Node.js และ Asynchronous Programming

อ่านประกอบจากสรุปหน้า 10–11 | เฉลยอยู่หลังโจทย์ทั้ง 15 ข้อของบทนี้

ข้อ 1 ถึง 10 ประเด็นตรงกับ pretest

5.1 Node.js คืออะไร ใช้ V8 ทำอะไร และต่างจาก browser หรือ database อย่างไร

5.2 Event loop มีบทบาทอย่างไรในการจัดการงาน asynchronous ของ Node.js

5.3 Non-blocking I/O มีข้อดีอย่างไรสำหรับ web server ที่รอฐานข้อมูลบ่อย

5.4 Promise คืออะไรและมีสถานะใดบ้าง

5.5 async/await ช่วยอะไร และ await หยุดทั้ง Node.js process หรือไม่

5.6 CommonJS นำเข้า module ด้วยคำสั่งใด ยกตัวอย่างนำเข้า HTTP module

5.7 npm มีหน้าที่อะไรในโครงการ Node.js

5.8 http.createServer() ใช้ทำอะไร callback ได้ req กับ res ไปใช้ต่างกันอย่างไร

5.9 ควรเก็บ database password ไว้ที่ใด และเพราะเหตุใดไม่ควรใส่ใน source code

5.10 เมื่อ await Promise ที่ rejected จะเกิดอะไร และควรจัดการอย่างไร

ข้อ 11 ถึง 15 ฝึกประยุกต์และเชื่อมความเข้าใจ

5.11 เปรียบเทียบงานอ่านไฟล์กับงานวนคำนวณหนักบน thread หลัก เหตุใด Node.js อาจตอบช้าในงานหลัง

5.12 async function f(){return 5;} เมื่อเรียก f() ได้ตัวเลข 5 โดยตรงหรือไม่ และ await f() ได้อะไร

5.13 พิจารณา async function f(){console.log("A"); await Promise.resolve(); console.log("B");} f(); console.log("C"); ลำดับข้อความคืออะไร

5.14 ถ้า catch พิมพ์ error แล้ว throw error ต่อ ผู้เรียกยังต้องจัดการข้อผิดพลาดหรือไม่

5.15 อธิบายโค้ด http.createServer((req,res)=>{res.writeHead(200);res.end("Hello");}).listen(3000)

เฉลยท้ายบทที่ 5 ข้อ 1 ถึง 5

ทวนจากสรุปหน้า 10–11 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 5.1

แนวตอบ: Node.js เป็น runtime สำหรับรัน JavaScript นอก browser โดยใช้ V8 เป็น engine ประมวลผล JavaScript ใช้สร้าง server และเครื่องมือได้ แต่ไม่ได้เป็น browser ที่ render หน้าเว็บหรือเป็นฐานข้อมูลเอง โปรแกรม Node.js สามารถติดต่อฐานข้อมูลอื่นได้

ประเด็นที่ควรครบ: runtime • V8 • นอก browser

ข้อ 5.2

แนวตอบ: Event loop ประสานการรันงานที่พร้อม เช่น callback จากงาน I/O
ทำให้โปรแกรมรับและจัดการงานต่อเนื่องระหว่างรอผลภายนอก เมื่อการทำงาน JavaScript
ปัจจุบันปล่อยการควบคุมจึงมีโอกาสรันงานต่อไป งาน CPU หนักบน thread หลักยังทำให้การตอบสนองช้าได้

ประเด็นที่ควรครบ: งานพร้อม • callback/I/O • ไม่ขจัด CPU blocking

ข้อ 5.3

แนวตอบ: ระหว่างรอฐานข้อมูล โปรแกรมสามารถจัดการคำขอหรืองานอื่นแทนที่จะหยุดรอทีละงาน
เมื่อผลพร้อมจึงทำขั้นตอนต่อ ช่วยรองรับงานที่มีช่วงรอจำนวนมาก แต่ไม่ได้ทำให้ทุกคำสั่ง JavaScript
ประมวลผลพร้อมกันหรือไม่ต้องจัดการ error

ประเด็นที่ควรครบ: รอ I/O • รับงานอื่น • กลับมาจัดผล

ข้อ 5.4

แนวตอบ: Promise เป็นตัวแทนผลของงานที่อาจยังไม่เสร็จ เริ่มเป็น pending เมื่อสำเร็จเป็น fulfilled พร้อมค่า หรือเมื่อไม่สำเร็จเป็น rejected พร้อมเหตุผล จากนั้นโค้ดที่รอผลหรือผูก handler จะจัดการผลตามสถานะ

ประเด็นที่ควรครบ: ผลในอนาคต • pending • fulfilled/rejected

ข้อ 5.5

แนวตอบ: async/await ช่วยเขียนการรอ Promise ให้ดูเป็นลำดับที่อ่านง่าย async function คืน Promise ส่วน await พักการทำงานของฟังก์ชันนั้นจนผลพร้อม ไม่ใช่หยุดทั้ง process
อย่างไรก็ดีคำสั่งคำนวณหนักที่รันก่อนหรือหลัง await ยังอาจบล็อก thread หลัก

ประเด็นที่ควรครบ: อ่านเป็นลำดับ • Promise • พักเฉพาะฟังก์ชัน

เฉลยท้ายบทที่ 5 ข้อ 6 ถึง 10

ทวนจากสรุปหน้า 10–11 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 5.6

แนวตอบ: ใช้ require() เช่น const http = require("node:http"); เพื่ออ้างฟังก์ชันจาก HTTP module ของ Node.js จากนั้นเรียก http.createServer() ได้ เป็นรูปแบบ CommonJS ตามคำถาม ไม่ใช่ importCSS หรือ include

ประเด็นที่ควรครบ: require() • module • node:http

ข้อ 5.7

แนวตอบ: npm เป็นเครื่องมือจัดการ package และ dependency ช่วยติดตั้งและจัดการไลบรารีที่โครงการใช้ รวมถึงเรียก scripts ที่กำหนดในโครงการ ไม่ใช่ฐานข้อมูล SQL หรือ web server
สำเร็จรูปที่ทำงานแทนโปรแกรมทั้งหมด

ประเด็นที่ควรครบ: package • dependency • โครงการ

ข้อ 5.8

แนวตอบ: ใช้สร้าง HTTP server โดยกำหนด callback สำหรับคำขอที่เข้ามา req แทนข้อมูล request ส่วน res ใช้กำหนดและส่ง response เช่นสถานะ header และ body หลังสร้าง server ต้องเริ่มฟังการเชื่อมต่อด้วย listen ตามการใช้งาน

ประเด็นที่ควรครบ: สร้าง server • req คำขอ • res คำตอบ

ข้อ 5.9

แนวตอบ: ควรใช้ environment variable หรือ secret manager พร้อมจัดสิทธิ์เหมาะสม เพราะรหัสที่ hard-code อาจรั่วผ่าน repository หรือไฟล์ที่เผยแพร่ การเก็บใน .env ก็ต้องไม่ commit ไฟล์ความลับ และไม่ส่งรหัสไป client โดยไม่จำเป็น

ประเด็นที่ควรครบ: environment/secret manager • ไม่เผยแพร่ • ไม่ commit ความลับ

ข้อ 5.10

แนวตอบ: await จะแสดงความล้มเหลวเป็นการโยน error ที่จุดรอผล จึงใช้ try...catch ครอบ await
หรือให้ผู้เรียกจัดการ Promise ที่ reject ต่อด้วย await/catch หรือ .catch() ต้องมีจุดรับผิดชอบ error ไม่ละเลย rejection

ประเด็นที่ควรครบ: โยน error • try/catch • ผู้เรียกจัดการได้

เฉลยท้ายบทที่ 5 ข้อ 11 ถึง 15

ทวนจากสรุปหน้า 10–11 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 5.11

แนวตอบ: งานอ่านไฟล์แบบ asynchronous เปิดโอกาสให้จัดการงานอื่นระหว่างรอ ส่วนลูปคำนวณหนักเป็น JavaScript ที่ยังครอง thread หลัก ทำให้ event loop ไม่มีโอกาสเดินหน้าจนโค้ดยอมคืนการควบคุม การใช้ Node.js จึงไม่ได้ป้องกันการ blocking ของงาน CPU ทุกแบบ

ประเด็นที่ควรครบ: I/O รอ • CPU ครอง thread • event loop ล่าช้า

ข้อ 5.12

แนวตอบ: f() คืน Promise ที่ fulfilled ด้วยค่า 5 ไม่ใช่ตัวเลข 5 โดยตรง เมื่อ await f() ภายในบริบทที่ใช้ await ได้ จะได้ผลลัพธ์ 5 หลักนี้เกิดจาก async function คืน Promise เสมอ

ประเด็นที่ควรครบ: f() เป็น Promise • await ได้ 5 • async

ข้อ 5.13

แนวตอบ: ลำดับคือ A, C, B ตอนเรียก f จะรันจนถึง await แล้วพักส่วนที่เหลือ โค้ดภายนอกจึงพิมพ์ C ก่อน จากนั้นงานต่อเนื่องหลัง await จึงพิมพ์ B แม้ Promise.resolve พร้อมผลแล้ว await ก็ไม่รันส่วนต่อเนื่องทันทีในลำดับ synchronous เดิม

ประเด็นที่ควรครบ: A C B • พักที่ await • โค้ดภายนอกทำต่อ

ข้อ 5.14

แนวตอบ: ยังต้องจัดการ เพราะ throw ต่อจาก catch ส่งความล้มเหลวออกไป หากเป็น async function จะทำให้ Promise ที่คืนกลับ reject ผู้เรียกต้อง await ใน try...catch หรือใช้ .catch() ตามเหมาะสม การบันทึก log ไม่เท่ากับคืนผลสำเร็จ

ประเด็นที่ควรครบ: throw ส่งต่อ • Promise reject • caller รับผิดชอบ

ข้อ 5.15

แนวตอบ: สร้าง HTTP server พร้อม handler เริ่มรับการเชื่อมต่อที่พอร์ต 3000 เมื่อรับ request จะกำหนดสถานะ 200 แล้วจบ response ด้วย body เป็น Hello ตัวอย่างนี้ไม่ได้แยก path จึงตอบแบบเดียวใน handler และไม่ได้เชื่อมฐานข้อมูลให้เอง

ประเด็นที่ควรครบ: server • port 3000 • status 200/body Hello

บทที่ 6 โจทย์เรื่อง ฐานข้อมูลเชิงสัมพันธ์และ SQL

อ่านประกอบจากสรุปหน้า 12–14 | เฉลยอยู่หลังโจทย์ทั้ง 15 ข้อของบทนี้

ข้อ 1 ถึง 10 ประเด็นตรงกับ pretest

ข้อมูลสำหรับข้อที่อ้างตัวอย่าง: users(id,name) = (1,Ann),(2,Bee),(3,Cat); orders(id,user_id,total) = (101,1,100),(102,1,200),(103,2,50) ให้เริ่มจากข้อมูลนี้ใหม่ในแต่ละข้อ

6.1 Relational database จัดข้อมูลอย่างไร อธิบาย table row column โดยใช้ข้อมูลผู้ใช้เป็นตัวอย่าง

6.2 Primary Key มีหน้าที่อะไร ถ้าชื่อผู้ใช้ซ้ำกันควรใช้ชื่อเป็น PK หรือไม่

6.3 Foreign Key ใช้ทำอะไร อธิบาย orders.user_id ที่อ้าง users.id

6.4 คำสั่งใดใช้ดึงข้อมูล เขียน SQL เพื่ออ่าน name ของผู้ใช้ id เท่ากับ 1

6.5 WHERE ทำหน้าที่อะไร และถ้า UPDATE หรือ DELETE ไม่มี WHERE อาจเกิดอะไร

6.6 INNER JOIN คืนข้อมูลแบบใด อธิบายด้วย users ที่มี Cat แต่ Cat ไม่มี order

6.7 GROUP BY ใช้ร่วมกับ COUNT หรือ SUM อย่างไร ยกตัวอย่างนับ orders ต่อ user_id

6.8 A ใน ACID คืออะไร อธิบายผ่านธุรกรรมโอนเงินสองบัญชี

6.9 Parameterized query ช่วยลด SQL injection ได้อย่างไร ต่างจากการต่อข้อความ input เข้า SQL อย่างไร

6.10 Index มีข้อดีและต้นทุนอะไร เหตุใดไม่ควรสร้างโดยคิดว่าทุก query จะเร็วเท่ากัน

ข้อ 11 ถึง 15 ฝึกประยุกต์และเชื่อมความเข้าใจ

6.11 ใช้ข้อมูลตัวอย่างในบทนี้ ระบุ PK, FK และความสัมพันธ์ระหว่าง users กับ orders

6.12 เขียน INNER JOIN ให้แสดงชื่อผู้ใช้ หมายเลข order และ total จากข้อมูลตัวอย่าง พร้อมบอกผลลัพธ์

6.13 เขียน query นับจำนวน orders และรวม total ต่อ user_id จากข้อมูลตัวอย่าง พร้อมบอกผลลัพธ์

6.14 เขียน SQL เพิ่มผู้ใช้ 4 ชื่อ Dan แก้ชื่อ id 4 เป็น Danny แล้วลบผู้ใช้ id 4

6.15 อธิบาย ACID ทั้งสี่ตัว และหน้าที่ของ BEGIN COMMIT ROLLBACK

เฉลยท้ายบทที่ 6 ข้อ 1 ถึง 5

ทวนจากสรุปหน้า 12–14 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 6.1

แนวตอบ: จัดข้อมูลเป็นตารางหรือ table โดยแต่ละ row แทนรายการ เช่นผู้ใช้หนึ่งคน และ column แทนคุณสมบัติ เช่น id กับ name ตาราง users จึงมีหลายแถวตามจำนวนผู้ใช้ และอาจเชื่อมกับตารางอื่นด้วยคีย์ SQL เป็นภาษาที่ใช้จัดการข้อมูล ไม่ใช่ชื่อของตาราง

ประเด็นที่ควรครบ: table • row รายการ • column คุณสมบัติ

ข้อ 6.2

แนวตอบ: PK ใช้ระบุแถวให้ไม่ซ้ำและระบุได้แน่นอน หาก name อาจซ้ำ ไม่เหมาะจะใช้ name เพียงอย่างเดียวเป็น PK ควรใช้รหัสที่ไม่ซ้ำ เช่น id หรือคีย์ที่ออกแบบตามเงื่อนไขข้อมูล คีย์อาจประกอบจากหลายคอลัมน์ร่วมกันได้

ประเด็นที่ควรครบ: ระบุแถว • ไม่ซ้ำ • เลือก id

ข้อ 6.3

แนวตอบ: FK เป็นคอลัมน์หรือชุดคอลัมน์ที่อ้างคีย์ของตารางที่เกี่ยวข้องเพื่อรักษาความสัมพันธ์ orders.user_id บอกว่า order เป็นของผู้ใช้ใดผ่าน users.id ค่าผู้ใช้คนเดียวกันปรากฏในหลาย orders ได้ จึงไม่จำเป็นต้องไม่ซ้ำเหมือน PK

ประเด็นที่ควรครบ: อ้างอิง • ความสัมพันธ์ • FK ซ้ำได้

ข้อ 6.4

แนวตอบ: ใช้ SELECT name FROM users WHERE id = 1; SELECT ระบุคอลัมน์ที่ต้องการ FROM ระบุตาราง และ WHERE ระบุเงื่อนไขแถว ตามข้อมูลตัวอย่างจะได้ Ann หากแถวนั้นมีอยู่ คำสั่งนี้อ่านโดยไม่แก้หรือลบข้อมูล

ประเด็นที่ควรครบ: SELECT • FROM • WHERE • Ann

ข้อ 6.5

แนวตอบ: WHERE ใช้กรองแถวตามเงื่อนไข ใน UPDATE หรือ DELETE ถ้าไม่ใส่ WHERE และไม่มีข้อจำกัดอื่นที่เกี่ยวข้อง อาจแก้หรือลบทุกแถวในตาราง เช่น UPDATE users SET name="X" ทำให้ทุกคนชื่อ X จึงควรตรวจขอบเขตเป้าหมายก่อน

ประเด็นที่ควรครบ: กรองแถว • ทุกแถวเมื่อไม่จำกัด • ตรวจเงื่อนไข

เฉลยท้ายบทที่ 6 ข้อ 6 ถึง 10

ทวนจากสรุปหน้า 12–14 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 6.6

แนวตอบ: INNER JOIN คืนเฉพาะแถวที่จับคู่ตาม ON ได้ เมื่อเชื่อม orders.user_id กับ users.id จะมี Ann และ Bee ตาม orders ที่มี ส่วน Cat ไม่ปรากฏเพราะไม่มีแถว order จับคู่ ไม่ใช่การคืนทุกแถวของ users เสมอ

ประเด็นที่ควรครบ: จับคู่ตาม ON • Cat ไม่ปรากฏ • ไม่ใช่ทุกแถว

ข้อ 6.7

แนวตอบ: GROUP BY แบ่งแถวเป็นกลุ่มก่อนคำนวณ aggregate เช่น SELECT user_id, COUNT(*) FROM orders GROUP BY user_id; นับจำนวน orders ในแต่ละกลุ่มผู้ใช้ ส่วน SUM(total) หาผลรวมยอดของแต่ละกลุ่ม GROUP BY ไม่ใช่การเชื่อมตารางแบบ JOIN

ประเด็นที่ควรครบ: จัดกลุ่ม • COUNT นับ • SUM รวม

ข้อ 6.8

แนวตอบ: A คือ Atomicity หมายถึงชุดงานสำเร็จทั้งหมดหรือยกเลิกทั้งหมด เช่นต้องหักเงิน A และเพิ่มเงิน B ให้สำเร็จร่วมกัน หากขั้นหนึ่งล้มเหลวต้องย้อนกลับเพื่อไม่ให้เกิดการหักเงินฝ่ายเดียว ใช้ transaction กำหนดขอบเขตงานนี้

ประเด็นที่ควรครบ: Atomicity • ทั้งหมดหรือไม่ทำ • โอนเงิน

ข้อ 6.9

แนวตอบ: Parameterized query ส่งโครงคำสั่งกับค่าข้อมูลแยกกัน เช่น WHERE name = ? แล้วส่งค่า name ให้ driver ทำให้ค่าถูกใช้เป็นข้อมูลแทนการนำไปต่อเป็นไวยากรณ์คำสั่งโดยตรง การต่อ string จาก input อาจทำให้ input เปลี่ยนความหมาย SQL ได้ ต้องใช้ placeholder ตาม driver

ประเด็นที่ควรครบ: แยกคำสั่งกับค่า • placeholder • ลด injection

ข้อ 6.10

แนวตอบ: Index ช่วยค้นหาข้อมูลในเงื่อนไขที่เหมาะสมได้เร็วขึ้นคล้ายสารบัญ แต่ใช้พื้นที่และต้องปรับปรุงเมื่อ INSERT/UPDATE/DELETE ประสิทธิภาพขึ้นกับ query และการเลือกใช้ของระบบ จึงไม่รับประกันว่าทุก query ใช้ index หรือเป็น O(1)

ประเด็นที่ควรครบ: ค้นหาบางแบบ • พื้นที่/เขียน • ไม่รับประกันทุก query

เฉลยท้ายบทที่ 6 ข้อ 11 ถึง 15

ทวนจากสรุปหน้า 12–14 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 6.11

แนวตอบ: users.id กับ orders.id เป็น PK ของแต่ละตาราง ส่วน orders.user_id เป็น FK อ้าง users.id ผู้ใช้หนึ่งคนมีหลาย orders ได้ เป็น one-to-many ตัวอย่าง Ann มีสอง orders คือ 101 และ 102 แต่ทั้งสองรายการมี id ของ order ไม่ซ้ำกัน

ประเด็นที่ควรครบ: PK สองตาราง • user_id FK • one-to-many

ข้อ 6.12

```
SELECT u.name, o.id, o.total
FROM users AS u
INNER JOIN orders AS o ON o.user_id = u.id;
```

ผลเป็น Ann,101,100; Ann,102,200; Bee,103,50 โดยไม่รับประกันลำดับแถวหากไม่มี ORDER BY ส่วน Cat ไม่มี order จึงไม่ปรากฏ

ประเด็นที่ควรครบ: ON เชื่อมคีย์ • สามแถว • Cat ไม่ปรากฏ

ข้อ 6.13

```
SELECT user_id, COUNT(*) AS order_count, SUM(total) AS amount
FROM orders
GROUP BY user_id;
```

ผู้ใช้ 1 มี order_count 2 และ amount 300 ผู้ใช้ 2 มี order_count 1 และ amount 50 เป็นการสรุปต่อกลุ่มผู้ใช้ ไม่ใช่แสดงรายการ order ทุกแถว

ประเด็นที่ควรครบ: COUNT • SUM • GROUP BY • ผล 2/300 และ 1/50

ข้อ 6.14

```
INSERT INTO users (id, name) VALUES (4, 'Dan');
UPDATE users SET name = 'Danny' WHERE id = 4;
DELETE FROM users WHERE id = 4;
```

ตัวอย่างสมมติว่ายังไม่มี id 4 และไม่มีข้อมูลอื่นอ้างถึงรายการนี้ คำสั่งทำงานเพิ่ม แก้ และลบตามลำดับ โดย WHERE จำกัดแถวเป้าหมายของสองคำสั่งหลัง

ประเด็นที่ควรครบ: INSERT • UPDATE SET/WHERE • DELETE WHERE

ข้อ 6.15

แนวตอบ: Atomicity คือสำเร็จหรือยกเลิกทั้งชุด Consistency คือเป็นไปตามกฎความถูกต้องที่กำหนด Isolation คือควบคุมผลกระทบระหว่างธุรกรรมที่ทำพร้อมกันตามระดับ isolation และ Durability คือผลที่ยืนยันแล้วคงอยู่ตามหลักประกันระบบ BEGIN เริ่ม transaction COMMIT ยืนยัน ROLLBACK ย้อนการเปลี่ยนแปลงของชุดงาน

ประเด็นที่ควรครบ: ACID สี่ด้าน • เริ่ม • ยืนยัน • ย้อนกลับ

บทที่ 7 โจทย์เรื่อง MongoDB และฐานข้อมูลแบบ Document

อ่านประกอบจากสรุปหน้า 15–17 | เฉลยอยู่หลังโจทย์ทั้ง 15 ข้อของบทนี้

ข้อ 1 ถึง 10 ประเด็นตรงกับ pretest

7.1 MongoDB เป็นฐานข้อมูลประเภทใด และข้อมูลต่างจากฐานข้อมูลเชิงสัมพันธ์ในภาพรวมอย่างไร

7.2 Document คืออะไร ยกตัวอย่างข้อมูลผู้ใช้หนึ่งรายการและระบุ field กับ value

7.3 Collection คืออะไร และเทียบกับแนวคิดใดใน SQL ได้โดยประมาณ

7.4 BSON ต่างจาก JSON อย่างไร ยกชนิดข้อมูลที่ทำให้เห็นความต่าง

7.5 CRUD ย่อมาจากอะไร และ C หมายถึงอะไร ยกคำสั่ง MongoDB ที่สัมพันธ์กัน

7.6 find() ใช้ทำอะไร และ db.users.find({age:{$gte:18}}) เลือกข้อมูลแบบใด

7.7 updateOne() ทำอะไร ส่วน filter กับ $set มีหน้าที่ต่างกันอย่างไร

7.8 ObjectId เกี่ยวข้องกับ _id อย่างไร และ _id ต้องเป็น ObjectId เสมอหรือไม่

7.9 Schema flexibility มีประโยชน์อะไร และหมายความว่าไม่ต้องตรวจข้อมูลหรือไม่

7.10 เมื่อใดควร embed ข้อมูลใน document เดียว และมีข้อควรระวังอะไร

ข้อ 11 ถึง 15 ฝึกประยุกต์และเชื่อมความเข้าใจ

7.11 เขียน MongoDB เพิ่มผู้ใช้ Ann อายุ 20 ค้นหาผู้ใช้อายุอย่างน้อย 18 แล้วแก้ Ann เป็นอายุ 21

7.12 หากมี Ann สอง document แล้วใช้ updateOne({name:"Ann"},{$set:{age:21}})
ทั้งสองรายการจะเปลี่ยนหรือไม่

7.13 Order ควรเก็บ customerId แบบ reference และที่อยู่จัดส่งแบบ embed ร่วมกันได้หรือไม่ เพราะเหตุใด

7.14 มีข้อมูลอายุ 20 กับ "20" ใน collection เดียวกัน ถือว่าเหมือนกันหรือไม่ และควรจัดการอย่างไร

7.15 เปรียบเทียบฐานข้อมูลเชิงสัมพันธ์กับ MongoDB และอธิบายว่าทำไมไม่ควรสรุปว่า MongoDB เร็วกว่าเสมอ

เฉลยท้ายบทที่ 7 ข้อ 1 ถึง 5

ทวนจากสรุปหน้า 15–17 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 7.1

แนวตอบ: MongoDB เป็น NoSQL ชนิด document database เก็บข้อมูลเป็น document ที่มี field และค่าซ้อนเป็น object หรือ array ได้ ส่วนฐานข้อมูลเชิงสัมพันธ์จัดข้อมูลหลักเป็นตารางแถวคอลัมน์แล้วใช้คีย์เชื่อมความสัมพันธ์ การเลือกใช้ขึ้นกับข้อมูลและรูปแบบการอ่านเขียน

ประเด็นที่ควรครบ: document database • ข้อมูลซ้อน • เทียบตาราง

ข้อ 7.2

แนวตอบ: Document คือหน่วยข้อมูลหนึ่งรายการ เช่น {name:"Ann",age:20} มี field name และ age โดยมี value เป็น "Ann" และ 20 ตามลำดับ Document อาจมี object หรือ array ซ้อนอยู่ และใน MongoDB มี _id ระบุแต่ละรายการ

ประเด็นที่ควรครบ: หนึ่งรายการ • key–value • _id

ข้อ 7.3

แนวตอบ: Collection เป็นกลุ่มของ document ภายใน database เช่น users เก็บ document ผู้ใช้หลายคน เทียบคร่าว ๆ กับ table ในฐานข้อมูลเชิงสัมพันธ์ แต่ document อาจมีโครงสร้างต่างกันและมีข้อมูลซ้อนได้ จึงเป็นเพียงการเทียบแนวคิด

ประเด็นที่ควรครบ: กลุ่ม document • table โดยประมาณ • ไม่เหมือนทุกด้าน

ข้อ 7.4

แนวตอบ: JSON เป็นรูปแบบข้อความแลกเปลี่ยนข้อมูล ส่วน BSON เป็นรูปแบบไบนารีที่ MongoDB ใช้แทน document และรองรับชนิดข้อมูลเพิ่มเติม เช่น Date กับ ObjectId BSON ไม่ใช่โพรโทคอลเครือข่ายหรือการเข้ารหัสความลับของ JSON

ประเด็นที่ควรครบ: ข้อความ/ไบนารี • Date/ObjectId • ชนิดเพิ่มเติม

ข้อ 7.5

แนวตอบ: CRUD คือ Create, Read, Update, Delete โดย C คือ Create หรือเพิ่มข้อมูลใหม่ ตัวอย่างตามลำดับคือ insertOne(), find(), updateOne() และ deleteOne() ไม่ควรตีความ C ว่า Compile หรือ Connect

ประเด็นที่ควรครบ: Create Read Update Delete • คำสั่งตรงหน้าที่

เฉลยท้ายบทที่ 7 ข้อ 6 ถึง 10

ทวนจากสรุปหน้า 15–17 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 7.6

แนวตอบ: find ค้นหา document ที่ตรง filter และคืน cursor สำหรับอ่านผลได้หลายรายการ ตัวอย่างเลือกผู้ใช้ที่ age เป็นค่าที่ตรงเงื่อนไขมากกว่าหรือเท่ากับ 18 โดย $gte คือ greater than or equal ไม่ใช่คำสั่งเพิ่มหรือลบ document

ประเด็นที่ควรครบ: ค้นหา • หลายรายการ • age ≥ 18

ข้อ 7.7

แนวตอบ: updateOne แก้หนึ่ง document ที่ตรง filter ส่วน filter ระบุว่าจะเลือกใคร เช่น {name:"Ann"} ส่วน update เช่น {$set:{age:21}} ระบุว่าจะกำหนด age เป็น 21 ไม่ใช่อ่านทุก document หรือสร้าง collection ใหม่อย่างเดียว

ประเด็นที่ควรครบ: หนึ่งรายการ • filter เลือก • $set กำหนดค่า

ข้อ 7.8

แนวตอบ: _id คือชื่อฟิลด์ตัวระบุ document ส่วน ObjectId เป็นชนิดค่าที่นิยมใช้เป็น _id
และมักถูกสร้างเมื่อไม่ได้ระบุไว้เอง _id ใช้ชนิดอื่นที่ระบบรองรับได้ จึงไม่ควรใช้คำว่า _id กับ ObjectId แทนกันทุกกรณี

ประเด็นที่ควรครบ: ชื่อ field • ชนิดค่า • ไม่จำเป็นเสมอ

ข้อ 7.9

แนวตอบ: Document ใน collection เดียวกันอาจมี field ต่างกันเมื่อเหมาะสม เช่นผู้ใช้บางคนมี phone บางคนไม่มี ช่วยรองรับข้อมูลหลากหลาย แต่ยังต้องตรวจชนิดและกฎสำคัญ เช่น age เป็นตัวเลข จึงยังควรมี validation และแบบแผนข้อมูลที่ชัด

ประเด็นที่ควรครบ: ยืดหยุ่น • ตัวอย่าง field ต่างกัน • validation ยังจำเป็น

ข้อ 7.10

แนวตอบ: เหมาะเมื่อข้อมูลมักอ่านร่วมกัน มีความสัมพันธ์ใกล้ชิด และขนาดหรือการเติบโตควบคุมได้
เช่นที่อยู่จัดส่งของ order ทำให้อ่านร่วมกันสะดวก
แต่ต้องระวังข้อมูลย่อยโตมากและข้อมูลซ้ำที่อาจต้องอัปเดตหลายจุด

ประเด็นที่ควรครบ: อ่านร่วมกัน • ขนาดควบคุมได้ • ข้อมูลซ้ำ/เติบโต

เฉลยท้ายบทที่ 7 ข้อ 11 ถึง 15

ทวนจากสรุปหน้า 15–17 | ตรวจใจความ ไม่จำเป็นต้องใช้คำเหมือนทุกคำ

ข้อ 7.11

```
db.users.insertOne({name:'Ann', age:20});
db.users.find({age:{$gte:18}});
db.users.updateOne({name:'Ann'}, {$set:{age:21}});
```

ตัวอย่างสมมติว่า Ann ระบุคนที่ต้องการได้ หากมีชื่อซ้ำและต้องเลือกคนแน่นอนควรใช้ _id ใน filter

ประเด็นที่ควรครบ: insertOne • find/$gte • updateOne/$set

ข้อ 7.12

แนวตอบ: ไม่เปลี่ยนทั้งสอง เพราะ updateOne แก้เพียงหนึ่งรายการที่ตรง filter
และไม่ควรเดาว่าจะเป็นคนใดเมื่อไม่ได้ระบุตัวตนแน่นอน ถ้าต้องการเจาะจงคนให้ใช้ _id ของคนนั้นในเงื่อนไข
หากไม่พบและไม่ได้เปิด upsert จะไม่เพิ่มรายการใหม่

ประเด็นที่ควรครบ: หนึ่งรายการ • ใช้ _id เจาะจง • ไม่พบไม่เพิ่มโดยปริยาย

ข้อ 7.13

แนวตอบ: ทำร่วมกันได้ customerId อ้างถึงข้อมูลลูกค้าปัจจุบันที่ใช้ร่วมหลาย order ส่วนที่อยู่จัดส่งแบบ embed เป็น snapshot ณ เวลาสั่งซื้อ จึงเก็บประวัติได้แม้ลูกค้าย้ายบ้าน
เป็นการเลือกแบบจำลองให้ตรงการอ่านและอายุของข้อมูลแต่ละส่วน

ประเด็นที่ควรครบ: ใช้ร่วมกันได้ • ข้อมูลปัจจุบัน • snapshot ประวัติ

ข้อ 7.14

แนวตอบ: ไม่เหมือนกัน เพราะ 20 เป็น number แต่ "20" เป็น string การเปรียบเทียบ คำนวณ และกรองอาจให้ผลต่างกัน ควรกำหนดชนิดที่ต้องการ ตรวจและแปลง input อย่างเหมาะสม รวมทั้งใช้ validation ตามความจำเป็น ความยืดหยุ่นไม่รับประกันคุณภาพข้อมูล

ประเด็นที่ควรครบ: ชนิดต่างกัน • query/คำนวณต่าง • ตรวจชนิด

ข้อ 7.15

แนวตอบ: เชิงสัมพันธ์เก็บตารางและเชื่อมผ่านคีย์ เหมาะกับข้อมูลและความสัมพันธ์ที่ต้องรักษากฎชัด MongoDB เก็บ document ที่จัดรูปแบบให้ตรงการอ่านของแอปและรองรับโครงสร้างหลากหลาย ความเร็วขึ้นกับ query, index, ปริมาณ และรูปแบบข้อมูล ทั้งสองมีความสามารถด้าน index และธุรกรรมตามระบบ จึงเลือกจากงานจริง

ประเด็นที่ควรครบ: ตาราง/เอกสาร • access pattern • ไม่มีผู้ชนะทุกกรณี

ทางลัดฝึกก่อนเข้าห้องสอบ

หากทำไม่ครบ 105 ข้อ ให้เลือกโจทย์รวมความเข้าใจต่อไปนี้ แล้วกลับไปทวนข้อย่อยในบทที่ยังอธิบายไม่คล่อง

บท	ข้อที่เลือกฝึก	เหตุผล
1	1.3 และ 1.7	เชื่อม request/response/render และ stateless
2	2.1 และ 2.12	แยกหน้าที่เว็บและอธิบาย DOM event
3	3.11 และ 3.12	ตรวจ const และผลของ map/push
4	4.9 และ 4.11	แยก routing กับ props/state
5	5.3 และ 5.10	งานที่ต้องรอและการจัดการ error
6	6.12 และ 6.13	JOIN และ GROUP BY พร้อมผลลัพธ์
7	7.11 และ 7.13	CRUD และเหตุผลเลือก embed/reference

ก่อนส่งกระดาษคำตอบ

อ่านคำสั่งโจทย์อีกครั้ง ถ้าถามเปรียบเทียบต้องตอบทั้งสองด้าน ถ้าถามผลโค้ดต้องบอกผลสุดท้ายและเหตุผล ถ้าถามออกแบบต้องอ้างลักษณะข้อมูลหรือการใช้งาน ถ้าเขียน keyword ได้แต่ยังเรียงเป็นคำอธิบายไม่ได้ ให้ฝึกพูดคำตอบสั้น ๆ ก่อนเขียน

แหล่งอ้างอิงของแนวคำถาม

ภาพ pretest ชุดที่ 1–7 ที่ผู้ใช้ส่ง และสรุปสอบเว็บแอปพลิเคชัน_บท1-7.pdf ฉบับ 20 หน้า เลขข้อรูปแบบ 6.12 หมายถึงบท 6 ข้อ 12 และเลขหน้าอ่านประกอบในแต่ละบทอ้างอิง PDF สรุปดังกล่าว